Desarrollo de un sistema de reservas con agentes

Informe de resolución — Challenge IA

Dato	Valor
Autor (git)	altamiranoaxel77
Fecha del informe	2026-11-05
Rama / commit base del informe	main / adb7542
Agente	Claude Code 2.1.305 (extensión de VS Code, Windows 10)
Modelo	claude-opus-5-5 (Claude Opus 5.5)
Lenguaje y almacenamiento	Python 3.14 (stdlib) + SQLite · CLI · pytest 9.2.0
Resultado	148 pruebas automáticas pasan · mutantes detectados 9/9

Este informe resume y enlaza la evidencia del repositorio. La fuente de verdad son los archivos versionados (README.md y docs/); las capturas son un complemento visual y no reemplazan las pruebas reproducibles.

Contenido

- 1. Consigna y problema
- 2. Especificación
- 3. Solución técnica
- 4. Preparación del agente
- 5. Tres episodios de delegación
- 6. Pruebas, auditoría y verificación
- 7. Capturas de funcionamiento
- 8. Cómo reproducir
- 9. Limitaciones y cumplimiento de la rúbrica
- Anexo: historial de commits

1. Consigna y problema

La consigna pide construir, **con un agente de programación que actúe sobre el repositorio con herramientas reales**, una solución local y verificable para la coordinación de un laboratorio que hoy registra pedidos en mensajes y una planilla. Dos fallas motivan el trabajo: la misma unidad se confirma para horarios superpuestos, y una cancelación sigue bloqueando el horario.

El inventario está fijo: cada ID es una unidad física distinta.

ID	Equipo
MIC-01	Microscopio digital
MIC-02	Microscopio digital
OSC-01	Osciloscopio
MUL-01	Multímetro
FUE-01	Fuente de alimentación

La entrega se evalúa por la evidencia del repositorio: especificación (25 %), preparación y uso del agente (25 %), funcionamiento y pruebas (30 %) y trazabilidad con tres episodios de delegación (20 %).

2. Especificación

Documento completo: `docs/especificacion.md` (versión 1.2). Su evolución v1 → v1.1 → v1.2 se recupera con `git log -p docs/especificacion.md` y está resumida en su §9.

Requisitos funcionales

ID	Requisito
R1	Cargar y mostrar el inventario precargado.
R2	Informar si una unidad está disponible en un intervalo y, si no, con qué reservas activas choca.
R3	Crear una reserva (unidad, solicitante, inicio, fin) con ID R-NNNN y estado activa.
R4	Cancelar por ID conservando ID y datos; estado cancelada, se registra cancelada_en y se libera el intervalo.
R5	Persistir reservas y estados; recuperarlos al reiniciar la aplicación.
R6	Listar reservas activas y canceladas, con filtros por unidad y estado.

Reglas de negocio centrales

- Unidad conocida y exacta (RN1); solicitante no vacío ni solo espacios (RN2); fecha y hora válidas en formato YYYY-MM-DD HH:MM (RN3); inicio estrictamente anterior al fin (RN4).
- Intervalos **semiabiertos [inicio, fin)**: A y B se superponen si A.inicio < B.fin y B.inicio < A.fin (RN5). Por eso 10:00–11:00 y 11:00–12:00 son consecutivos y válidos (RN8).
- Se rechaza la superposición con reservas **activas** de la **misma** unidad (RN6); una cancelada nunca bloquea (RN7); unidades distintas pueden reservarse a la vez (RN9).
- La consulta de disponibilidad aplica **exactamente las mismas reglas** que la creación (RN10).
- Solo se cancela una reserva existente y activa (RN11).

Supuestos (lo no especificado, resuelto)

ID	Supuesto
S1	Hora local del laboratorio: America/Argentina/Buenos_Aires, UTC−3 fijo, independiente de la PC.
S2	Formato exacto YYYY-MM-DD HH:MM, dígitos ASCII, resolución de minutos.

ID	Supuesto
S3	Se rechaza reservar o consultar con inicio en el pasado; una reserva pasada sí se puede cancelar.
S4 / S5	Duración máxima 12 h; sin límite de anticipación.
S6	IDs R-0001… secuenciales que nunca se reutilizan, ni tras reiniciar datos.
S7	Solicitante: máx. 100 caracteres, sin caracteres de control; siempre ficticio.
S8	El 'ahora' es fijable con RESERVAS_AHORA para pruebas reproducibles.
S9	reiniciar-datos --confirmar borra todas las reservas; sin --confirmar no hace nada.
S10 / S11	Filtros de listar validan unidad/estado; rutas configurables; errores de almacenamiento sin traceback.

Hay **16 criterios de aceptación** (CA-01…CA-16) en formato dado/cuando/entonces; incluyen el ejemplo de la consigna: con MIC-01 reservado de 10:00 a 11:00 se rechaza 10:30–11:30, se permite 11:00–12:00 y MIC-02 de 10:00 a 11:00.

3. Solución técnica

Aplicación de consola en Python que usa solo la biblioteca estándar en tiempo de ejecución; pytest es la única dependencia de desarrollo y está fijada en `requirements-dev.txt`.

```
data/inventario.json      inventario inicial (versionado)
src/reservas/
  dominio.py              reglas puras: validaciones, se_superpone() [inicio, fin), conflictos()
  repositorio.py          SQLite: esquema, transacciones BEGIN IMMEDIATE, sin DELETE salvo reinicio
  servicio.py             casos de uso: inventario, disponibilidad, reservar, cancelar, listar
  cli.py, __main__.py     interfaz: python -m reservas <comando>
  errores.py, inventario.py
tests/                    148 pruebas pytest
tools/                    auditoría de mutantes, capturas, este informe
```

Decisiones técnicas principales

Decisión	Motivo
SQLite (sqlite3 de la stdlib)	Transacciones atómicas; AUTOINCREMENT no reutiliza IDs; sin servidor.
Una única función se_superpone()	Garantiza que reservar y disponibilidad usen la misma regla (RN10).
Chequeo + INSERT en la misma transacción BEGIN IMMEDIATE	Evita dobles confirmaciones entre la verificación y la escritura.
Cancelar = UPDATE de estado; nunca DELETE	Conserva el historial (R4).
'Ahora' inyectable	Pruebas deterministas que no caducan con el calendario.
CLI con códigos de salida 0 / 1 / 2	0 éxito · 1 error de negocio o validación ('ERROR:' en stderr) · 2 uso incorrecto.

Las diez decisiones con alternativas y consecuencias están en `docs/decisiones.md` (Parte A).

Persistencia y reinicio de datos

Las reservas se guardan en `data/reservas.db` (ignorado por git; configurable con `--db` o `RESERVAS_DB`). El comando `reiniciar-datos --confirmar` **borra todas las reservas**; sin la bandera no hace nada.

4. Preparación del agente

Elemento	Detalle
Harness	Claude Code 2.1.305 (extensión de VS Code, Windows 10). Se usó uno existente; no se construyó ninguno.
Modelo	claude-opus-5-5 (Claude Opus 5.5). Datos no expuestos por el harness (temperatura, ventana de contexto) no se informan.
Instrucciones	CLAUDE.md: reglas de dominio no negociables, procedimiento de persistencia, forma de trabajo por etapas con commit y registro en docs/progreso.md.
Subagentes	.claude/agents/: redactor-pruebas (escribe pruebas desde los CA sin leer src/), revisor-reglas (auditoría solo lectura), verificador (ejecuta y registra evidencia).
Permisos	.claude/settings.json — allow: pytest, CLI, git de lectura · ask: git commit/push, rm, pip install, reiniciar-datos · deny: .env, *.db, acceso web.
Contexto excluido	.venv/, bases de datos, .env, web; al redactor-pruebas se le excluyó src/.

Adaptación específica comprobable

- **Turnos consecutivos:** CLAUDE.md exige [inicio, fin) y una única se_superpone(). El revisor lo verificó con búsqueda en el código y lo cubren las pruebas test_ca_04_* y test_dominio.py.
- **Procedimiento de persistencia:** transacción BEGIN IMMEDIATE, sin DELETE salvo reinicio, cancelar por UPDATE. Lo cubren test_ca_11_* y test_ca_14_*; los mutantes M6 y M8 lo confirman.
- **Retención del progreso:** docs/progreso.md se actualizó y versionó al cierre de cada etapa.

Controles humanos aplicados

- El autor eligió stack, interfaz y enfoque de agentes y aprobó el plan antes de empezar.
- Checkpoint 1: confirmó sin cambios los supuestos S3, S4 y RN1 de la especificación v1.
- Aprobó cada commit (rechazó uno por error y luego lo confirmó explícitamente).
- Decidió aceptar, corregir o rechazar en los tres episodios (sección 5).

Nota honesta: los subagentes se definieron en la misma sesión en que se usaron; el harness aún no los registraba como tipos propios, por lo que se lanzaron como agentes genéricos con la orden de leer y seguir su archivo de definición. Detalle en `docs/agente.md`.

5. Tres episodios de delegación

Cada episodio sigue la cadena exigida: objetivo → contexto e instrucción → resultado del agente → decisión humana → evidencia verificable.

	Episodio 1: implementación	**Episodio 2: pruebas**	**Episodio 3: revisión**
Delegado a	Agente principal	Subagente redactor-pruebas	Subagente revisor-reglas (solo lectura)
Objetivo	R1–R6 y RN1–RN11 de la especificación v1	Pruebas de CA-01…CA-15 desde la especificación	Auditar el código contra reglas y supuestos
Contexto / instrucción	Especificación, interfaz y CLAUDE.md ('una única se_superpone()', 'chequeo e INSERT en la misma transacción')	Especificación e interfaz; **sin acceso a src/**; 'si una prueba falla, no la arregles: reportala'	Todo el repo sin escritura; checklist de 7 ítems; 'no inventes problemas'
Resultado del agente	Código completo; prueba rápida coincide con el ejemplo de la consigna	114 pruebas; **3 fallaban**: tuple en lugar de list según el contrato; 11 ambigüedades	13 hallazgos (2 media, 11 baja); uno resultó solo parcialmente correcto al reproducirlo
Decisión humana	**Aceptar**	**Corregir la implementación** (no las pruebas) y aclarar IDs tras reinicio (spec v1.1, CA-16)	**Aceptar los 4 grupos** de correcciones (spec v1.2 + 23 pruebas de regresión)
Evidencia	commit 8e68a63	d76aeb4 → ed6385c, 1f29ccb; docs/evidencia/episodio2-*.md	7f502bd; docs/evidencia/episodio3-*.md

En todos los casos, lo informado por un subagente se comprobó re-ejecutando pytest o reproduciendo el caso antes de decidir; así se detectó, por ejemplo, que el hallazgo #7 del revisor era parcialmente incorrecto.

6. Pruebas, auditoría y verificación

Verificaciones exigidas por la consigna

Verificación exigida	Pruebas que la cubren
Creación válida	test_crear.py; test_ca_01_cli_reservar_salida_creada
Rechazo de superposición	test_superposicion.py (parcial, contiene, contenida, idéntica, solapes de 1 min)
Límites consecutivos	test_limites.py (incluye cruce de medianoche); test_dominio.py
Unidades distintas simultáneas	test_unidades.py::test_ca_05_*; test_ca_13_otra_unidad_disponible
Entradas inválidas	test_validaciones.py (31); test_ca_06_*; 12 casos por CLI con exit 1; test_revision.py
Cancelación que conserva y libera	test_cancelacion.py (CA-11, CA-12); test_dominio_conflictos_ignora_canceladas
Persistencia tras detener y reiniciar	test_persistencia.py: cada comando en un proceso nuevo sobre la misma BD

Auditoría de las pruebas y mutantes

Se leyeron los 11 archivos de pruebas y se corrigieron tres debilidades (un pytest.raises(Exception) demasiado amplio, falta de casos de medianoche/bisiesto, y la regla RN7 sin prueba a nivel de dominio). Luego `tools/auditoria/mutantes.py` inyectó 9 defectos, uno por vez, en una copia del proyecto: la primera corrida detectó 8/9 (sobrevivió M2) y, tras agregar `tests/test_dominio.py`, la segunda detectó **9/9**. Detalle en `docs/auditoria-pruebas.md`.

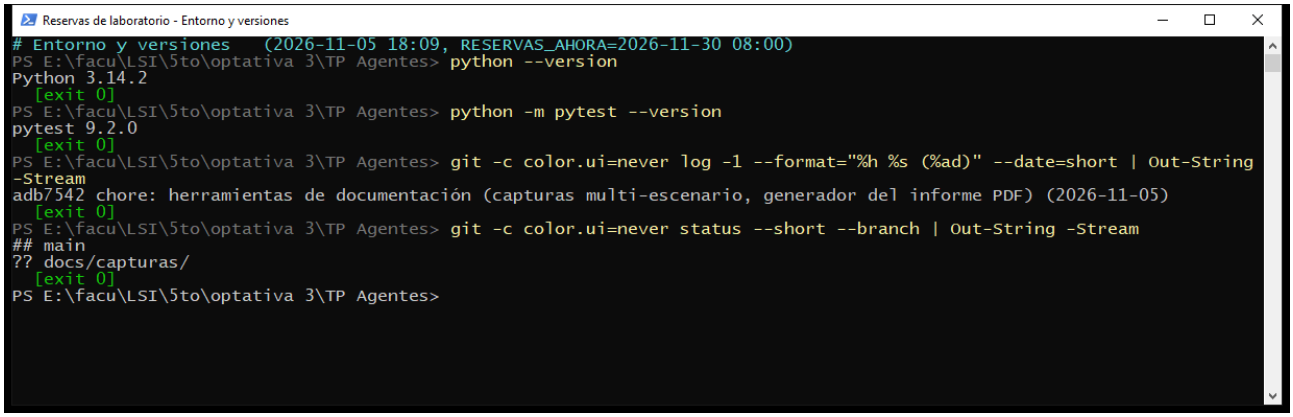
Resultados ejecutados

Verificación	Comando	Resultado
Suite completa (tras auditoría)	.venv/Scripts/python -m pytest -q	148 passed
Mutantes	.venv/Scripts/python tools/auditoria/mutantes.py	9/9 detectados
Persistencia manual (subagente verificador)	9 pasos, un proceso por paso (docs/verificacion.md)	9/9 OK (sobre 7f502bd)

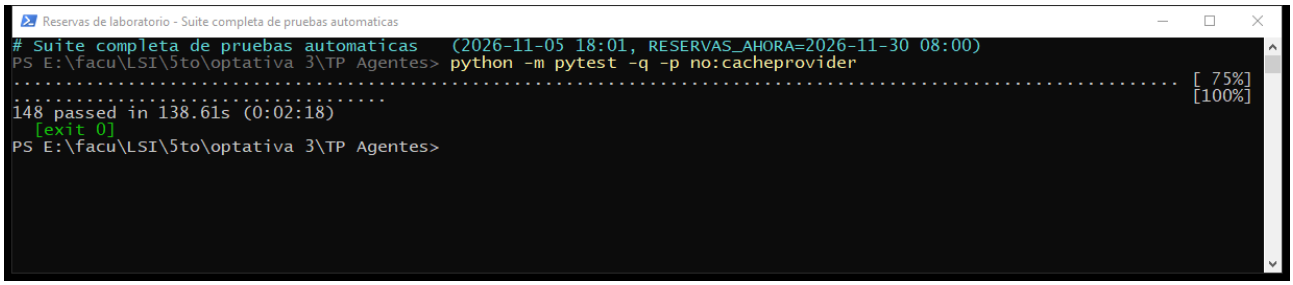
No se verificó: escrituras simultáneas de varios procesos, fallas físicas de disco, otros sistemas operativos.

7. Capturas de funcionamiento

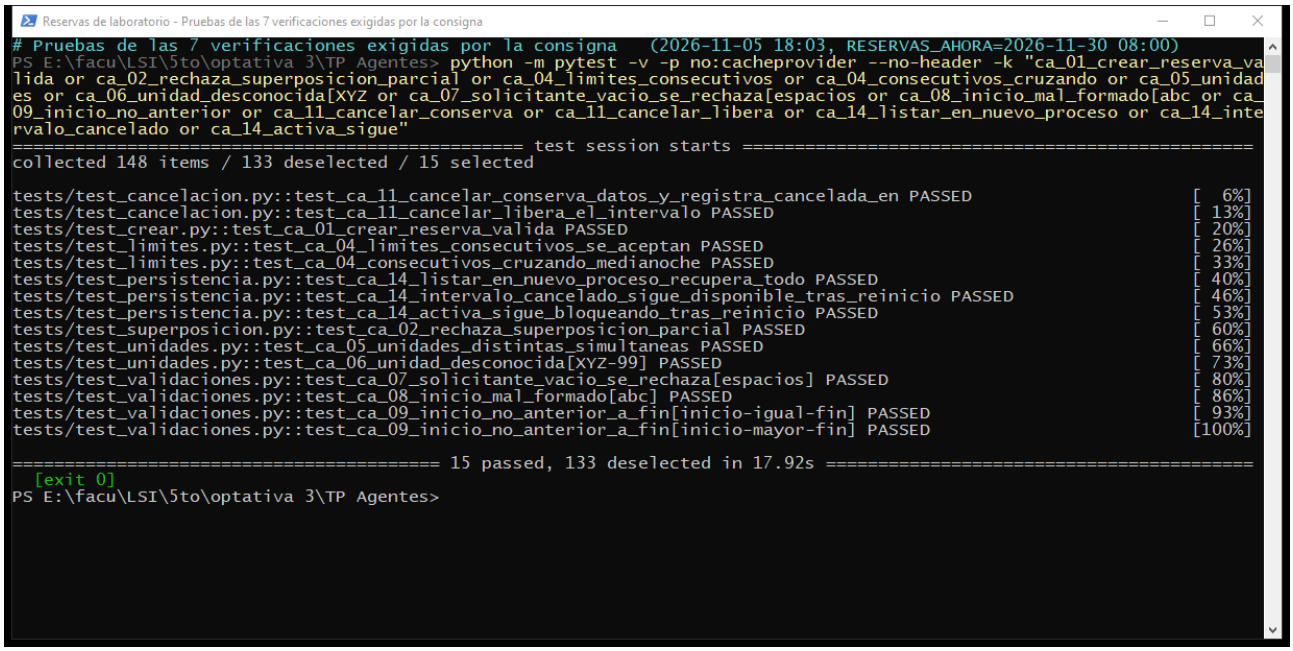
Capturas **reales** de ventanas de PowerShell generadas por `tools/capturas/capturar.ps1`: cada escenario se ejecuta en una ventana nueva, se captura con la API PrintWindow y se cierra. Las reservas usan una base de demostración en %TEMP% y RESERVAS_AHORA=2026-11-30 08:00. Descripción de cada una en `docs/capturas/README.md`.



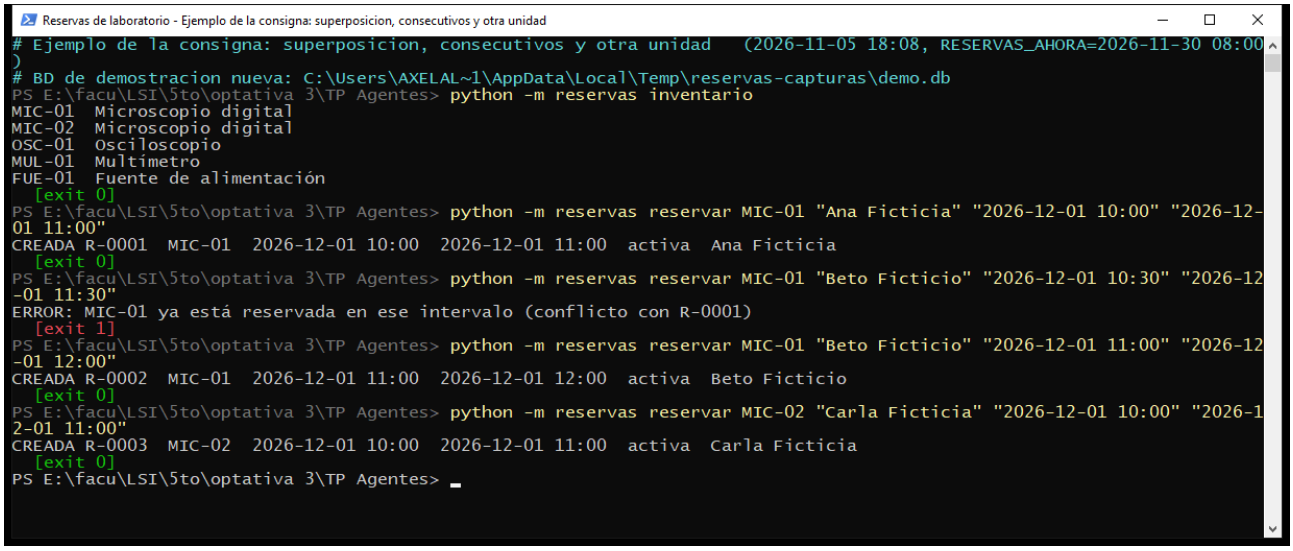
01-entorno.png — versiones de Python y pytest, commit y estado del árbol.
Reproducir: `capturar.ps1 -Solo 01`



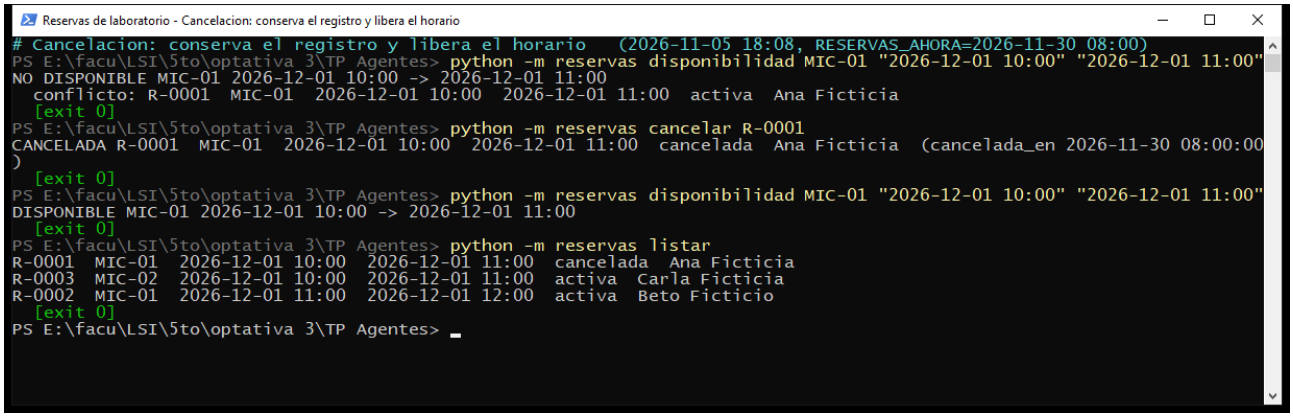
02-pytest-completo.png — suite completa de pruebas automáticas.
Reproducir: `python -m pytest -q`



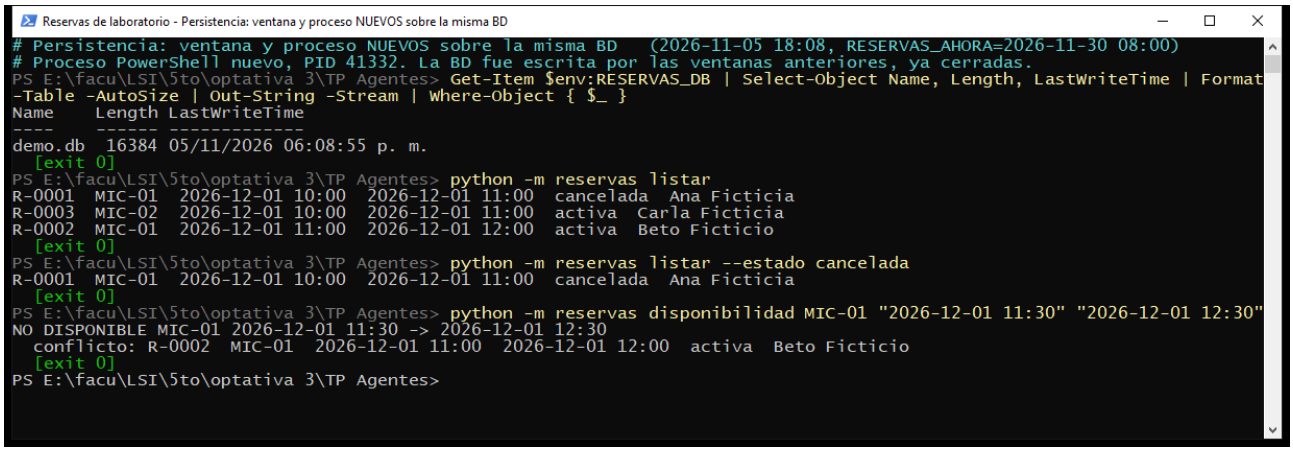
03-pytest-verificaciones-consigna.png — pruebas representativas de las 7 verificaciones exigidas.
Reproducir: `capturar.ps1 -Solo 03`



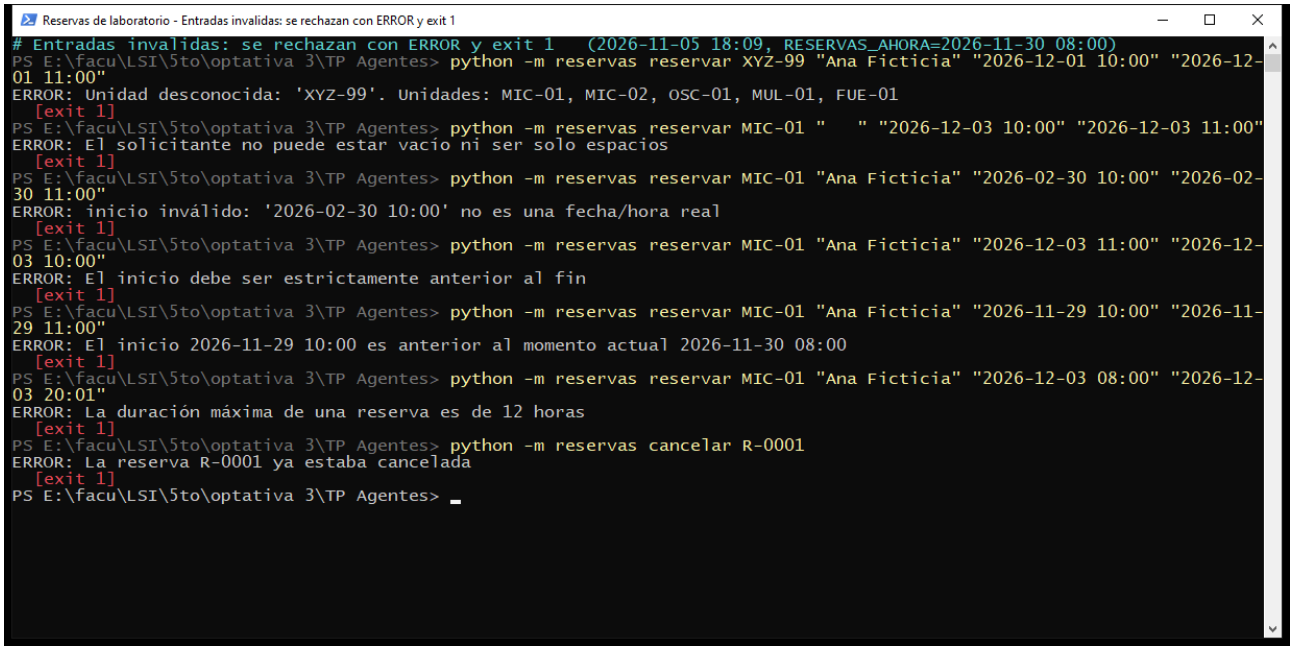
04-ejemplo-consigna.png — MIC-01 10–11 creada; 10:30–11:30 rechazada; 11–12 aceptada; MIC-02 10–11 aceptada.
Reproducir: `capturar.ps1 -Solo 04`



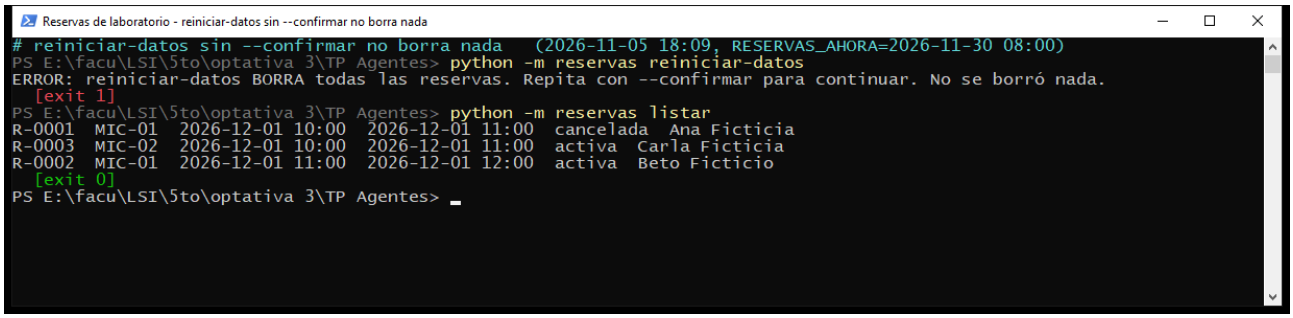
05-cancelacion.png — NO DISPONIBLE → cancelar R-0001 → DISPONIBLE; listar conserva la cancelada.
Reproducir: `capturar.ps1 -Solo 05`



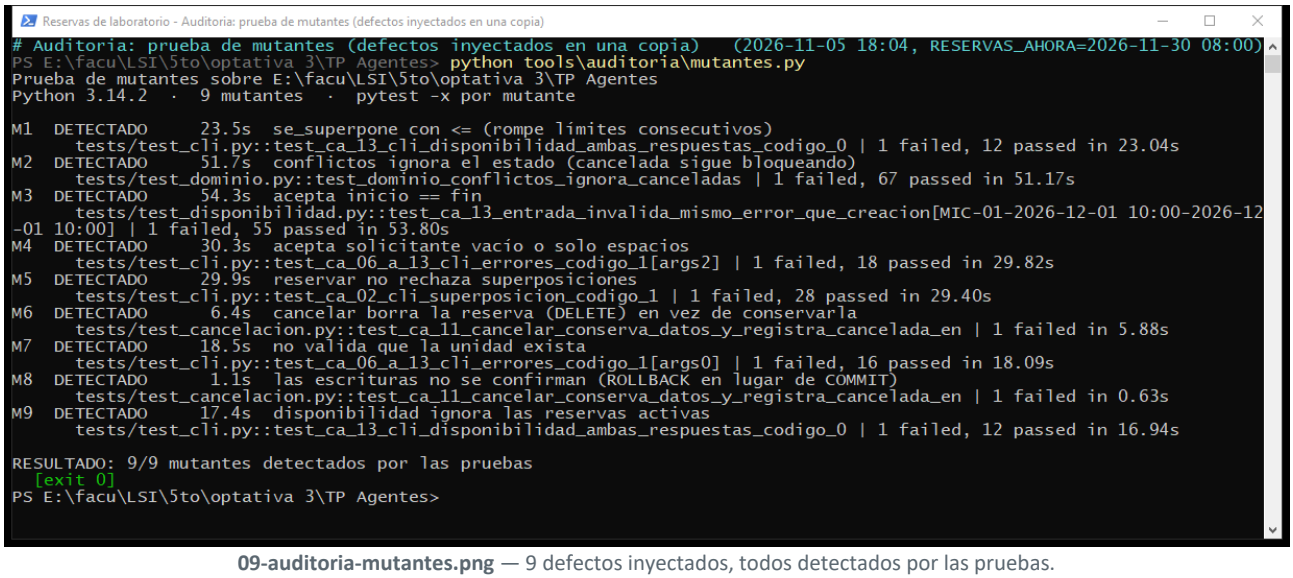
06-persistencia-nueva-ventana.png — proceso nuevo sobre la misma BD: las reservas activas y canceladas persisten.
Reproducir: `capturar.ps1 -Solo 06`



07-entradas-invalidas.png — unidad, solicitante, fecha, orden, pasado, duración y cancelación repetida: ERROR + exit 1.
Reproducir: `capturar.ps1 -Solo 07`



08-reiniciar-datos.png — reiniciar-datos sin --confirmar no borra nada.
Reproducir: `capturar.ps1 -Solo 08`



09-auditoria-mutantes.png — 9 defectos inyectados, todos detectados por las pruebas.
Reproducir: `python tools/auditoria/mutantes.py`

8. Cómo reproducir

Requisitos: Python 3.12 o superior y git. No se necesita red ni cuenta de modelo.

```
python -m venv .venv
.venv/Scripts/python -m pip install -r requirements-dev.txt -e .
.venv/Scripts/python -m pytest -v                        # pruebas automáticas
.venv/Scripts/python tools/auditoria/mutantes.py         # prueba de mutantes

# CLI (en PowerShell, BD de prueba y 'ahora' fijo):
$env:RESERVAS_DB = "$env:TEMP\prueba.db"; $env:RESERVAS_AHORA = "2026-11-30 08:00"
.venv/Scripts/python -m reservas reservar MIC-01 "Ana Ficticia" "2026-12-01 10:00" "2026-12-01 11:00"
.venv/Scripts/python -m reservas disponibilidad MIC-01 "2026-12-01 10:30" "2026-12-01 11:30"
.venv/Scripts/python -m reservas cancelar R-0001
.venv/Scripts/python -m reservas listar

# Capturas e informe (Windows):
powershell -ExecutionPolicy Bypass -File tools/capturas/capturar.ps1
python -m venv .venv-tools; .venv-tools/Scripts/python -m pip install -r tools/requirements-tools.txt
.venv-tools/Scripts/python tools/informe/generar_informe.py
```

9. Limitaciones y cumplimiento de la rúbrica

Limitaciones declaradas

- Sin pruebas de escrituras simultáneas desde varios procesos (fuera del alcance de la consigna).
- Verificado solo en Windows 10; en consolas no UTF-8 los acentos pueden verse mal (los datos no se alteran).
- Zona horaria UTC−3 fija: si Argentina volviera al horario de verano habría que ajustar ZONA_LABORATORIO.
- Sin edición/reprogramación de reservas ni reservas multi-unidad (fuera de alcance).

Cumplimiento por dimensión

Dimensión (peso)	Evidencia en el repositorio
Especificación (25 %)	docs/especificacion.md v1.2: R1–R6, RN1–RN11, S1–S11, CA-01…CA-16; historial v1→v1.2 con motivos en git.
Preparación y uso del agente (25 %)	CLAUDE.md, .claude/settings.json, .claude/agents/*.md, docs/agente.md, docs/progreso.md, docs/plan/.
Funcionamiento y pruebas (30 %)	148 pruebas; persistencia por subprocesos; docs/verificacion.md; docs/auditoria-pruebas.md (mutantes 9/9); capturas.
Trazabilidad y decisiones (20 %)	docs/decisiones.md (D1–D10 y 3 episodios), docs/evidencia/, commits enlazados.

Anexo: historial de commits

La evolución del trabajo se recupera en git; cada etapa terminó con un commit.

Commit	Mensaje
bd378e4	chore: preparación del agente
8db45b1	docs: especificación v1 e interfaz pública
674eadb	docs: especificación v1 aprobada por el autor sin cambios
8e68a63	feat: implementación de dominio, persistencia SQLite, servicio y CLI
d76aeb4	test: suite de pruebas CA-01..CA-15 escrita por el subagente redactor-pruebas
ed6385c	fix: Disponibilidad.conflictos devuelve list según el contrato
1f29ccb	docs: especificación v1.1 — IDs no se reutilizan tras reiniciar-datos (CA-16)
7f502bd	fix: hallazgos del revisor-reglas aceptados por el autor (Episodio 3)
554205d	docs: verificación, README, preparación del agente y registro de decisiones
ab1ee6f	test: auditoría de pruebas contra la consigna y prueba de mutantes
34e8743	chore: script de capturas reales de ventanas PowerShell (tools/capturas)
adb7542	chore: herramientas de documentación (capturas multi-escenario, generador del informe PDF)